

ChargeHub Socket 网络编程详解

从网络基础到本项目的 TCP、HTTP 与外部接口

对照作业目录当前源码

2026 年 9 月

摘要

本文按教科书顺序讲：先 IP/端口/TCP，再 Socket 与 Qt 封装，再讲本项目自己定的长度前缀 JSON，最后讲 HTTP/HTTPS 大屏和腾讯/高德等出站接口。读完应能回答：为什么用户端不直连数据库、粘包怎么切、`readyRead` 里为什么必须 `readAll`、天气请求为什么不走 8888。不粘贴 Key / SK / 密码。字段表见 `protocol/messages.md`。

目录

1	先建立地图：本项目里有几条「网」	2
2	网络基础	2
2.1	IP 地址：找到那台电脑	2
2.2	端口：找到那台电脑上的哪个程序	2
2.3	客户端与服务器	3
2.4	分层模型（够用即可）	3
3	TCP 为什么适合充电业务	3
3.1	TCP 提供什么	3
3.2	为什么不用 UDP	3
3.3	TCP 不提供「消息」	3
4	字节序、UTF-8 与长度前缀	4
4.1	大端与小端	4
4.2	UTF-8 JSON	4
4.3	一帧的样子	4
5	Socket 是什么：从 BSD 到 Qt	4
5.1	套接字 = 编程入口	4
5.2	阻塞 vs 事件驱动	5

6 本项目的 TCP 编程 (8888)	5
6.1 服务端：先听再接	5
6.2 客户端：拨号再发业务	5
6.3 接收循环为什么必须写成这样	5
7 应用层协议：我们自己的「HTTP」	6
7.1 请求信封	6
7.2 响应信封	6
7.3 请求-响应，以及服务器推送	6
7.4 鉴权发生在哪一层	6
7.5 幂等：网络编程里的业务问题	6
8 HTTP 与 HTTPS：另一套应用层	7
8.1 HTTP 报文长什么样	7
8.2 大屏：HTTP 只读	7
8.3 出站 WebService	7
8.4 DNS 与超时	7
9 连接生命周期与故障	7
9.1 正常关闭	7
9.2 异常断电	8
9.3 重连	8
9.4 一人两根线	8
10 地址怎么填：联调时的网络	8
11 对照源码：建议阅读顺序	8
12 把知识收成一张表	9

1 先建立地图：本项目里有几条「网」

ChargeHub 不是单机计算器，是至少两个进程通过网络说话：

通道	传输层	本项目用法
用户端 ↔ 管理端	TCP 端口 8888	自己设计的长度前缀 JSON（Socket 编程主体）
浏览器 ↔ 大屏	TCP 上的 HTTP 5000	Flask 只读 JSON，标准 Web
用户端/管理端 → 互联网	TCP 上的 HTTP/HTTPS	腾讯、高德瓦片、Open-Meteo 等

三条通道都建立在 **TCP** 上，但应用层协议不同：8888 是我们自己的二进制帧；5000 和腾讯是 HTTP 文本/JSON。操作系统只保证把字节按序送到，**不理解** JSON 或“LOGIN”。把字节解释成业务，是应用层（我们的 Protocol、Flask、浏览器）的事。

2 网络基础

2.1 IP 地址：找到那台电脑

IP 地址标识一台主机在某个网络里的位置。本项目常见三种：

127.0.0.1

本机回环。用户端和管理端在**同一台电脑**时填它。数据包不出网卡。

局域网地址，如 192.168.1.8、WSL 的 172.29.x.x

同一 Wi-Fi / 同一虚拟网里别人能 ping 到的地址。管理端底栏显示的就是这种。组员用户端必须填**值班那台**的局域网 IP，填自己的 127.0.0.1 会连到自己机器（往往没开管理端）。

公网 IP

校园网/家用路由后面通常没有真正的公网监听。本作业按局域网联调，不要求公网映射。

IPv4 是 32 位，点分十进制四段。本作业只使用 IPv4（QHostAddress::Any 实际听 IPv4 的 0.0.0.0）。

2.2 端口：找到那台电脑上的哪个程序

一台电脑可以同时跑浏览器、管理端、大屏。端口把它们分开：

- 端口是 16 位整数，范围 0–65535。
- 服务端绑定一个约定端口等别人来：本项目管理端 **8888**，大屏 **5000**。
- 客户端通常由操作系统分配临时端口（例如 49152 以上），你在登录页填的是**对方**的端口，不是自己的。

所以完整地址是 IP：端口。登录页 127.0.0.1:8888 的意思是：到本机上那个正在 **listen(8888)** 的进程。浏览器打开 **http://127.0.0.1:5000** 则是另一个进程。**不要用浏览器打开 8888**：浏览器讲 HTTP，管理端 8888 讲我们自己的帧，对不上。

2.3 客户端与服务端

- 服务器先启动、先 `listen`，被动等待。管理端必须先开。
- 客户端后启动、主动 `connect`。用户端、浏览器、烟测脚本都是客户端。

同一台机器可以既是 8888 的服务器，又是腾讯 HTTPS 的客户端：管理端相对用户端是服务器，相对 `apis.map.qq.com` 是客户端。角色取决于谁先听、谁去连。

2.4 分层模型（够用即可）

不必背全 OSI 七层。记住四层就够把本项目对上：

层	本项目对应	你改代码时碰不碰
应用层	JSON / HTTP / 腾讯 Webservice	碰：type、URL、Flask 路由
传输层	TCP（可靠字节流）	间接碰：端口、QTcpSocket
网络层	IP 路由、127.0.0.1 vs 局域网	联调时填对地址
链路/物理	网卡、Wi-Fi、虚拟网桥	一般不改代码

Qt 的 QTcpSocket 帮你走完传输层；Protocol 和 Flask 才是应用层。

3 TCP 为什么适合充电业务

3.1 TCP 提供什么

TCP（Transmission Control Protocol）在两台主机之间提供：

- 面向连接：先三次握手，双方确认后再传数据。对应 Qt 的 `connected` 信号。
- 可靠：丢包会重传，重复会丢弃，乱序会排好。应用层看到的是按发送顺序的字节。
- 双向字节流：两边都能随时 `write`。所以服务端能主动推 `PUSH_CHARGE`。

三次握手只需记住：客户端 `connect` 成功，说明连接已建立，可以发 `LOGIN`。四次挥手对应断开：用户端关掉窗口，管理端收到 `disconnected`。

3.2 为什么不用 UDP

UDP 不握手、不保证到达和顺序，适合游戏位置广播、视频。充电要：开充必须到达、结算不能乱序、推送最好别丢到「界面永远停住」（我们另外用轮询兜底）。所以业务通道选 TCP。地图瓦片虽然也是 TCP 上的 HTTP，失败只换下一张 URL，不改账。

3.3 TCP 不提供「消息」

这是 Socket 编程里最容易错的一点：

TCP 给你的是字节流，不是「一封信」。一次 `write` 可能被拆成两次 `readyRead`；两次 `write` 也可能粘在一次 `read` 里。这叫半包和粘包。

如果直接 `readAll()` 当一条 JSON，偶发会解析失败或把两单粘成一次。所以必须自己在字节流上划边界。本项目的划法：前面 4 字节告诉你后面 JSON 有多长。

4 字节序、UTF-8 与长度前缀

4.1 大端与小端

多字节整数在内存里有两种排法。网络约定用**大端**（高位字节先发出），又称网络字节序。x86/Windows/WSL 本机一般是小端，所以写长度时要转换：

```
qToBigEndian<quint32>(body.size(), out.data()); // 发送
qFromBigEndian<quint32>(buf.constData()); // 接收
```

Python 烟测与此完全一致：`struct.pack(">I", len(body))`，`>` 就是大端。

4.2 UTF-8 JSON

长度统计的是**字节**，不是汉字个数。「北京」UTF-8 是 6 字节。`QJsonDocument::Compact` 去掉空格，两端必须同一套编码，否则长度对、内容却解不出来。

4.3 一帧的样子

```
+-----+-----+
| 4 bytes | N bytes UTF-8 JSON |
| big-endian | {"type":"LOGIN", ...} |
| length = N | |
+-----+-----+
```

非法长度（0 或大于 1 MB）时，接收端清空缓冲：宁可变一次丢包，也不要永远等一个假长度。

5 Socket 是什么：从 BSD 到 Qt

5.1 套接字 = 编程入口

Socket 是操作系统提供的「网线插头」。服务端、客户端各拿一个，连上之后就能读写字节。经典 BSD 接口与本项目 Qt 的对应：

BSD / POSIX	Qt（本项目）	发生在哪
<code>socket()</code>	构造 <code>QTcpSocket</code> / <code>QTcpServer</code>	客户端 / 服务端
<code>bind()+listen()</code>	<code>QTcpServer::listen(Any, 8888)</code>	<code>adminserver/src/main.cpp</code>
<code>accept()</code>	<code>incomingConnection</code> 里 <code>setSocketDescriptor</code>	<code>tcpserver.cpp</code>
<code>connect()</code>	<code>QTcpSocket::connectToHost</code>	<code>client.cpp</code>
<code>send()/write()</code>	<code>socket->write(Protocol::pack(...))</code>	两端
<code>recv()/read()</code>	<code>readyRead</code> 里 <code>readAll()</code>	两端
<code>close()</code>	窗口关 / <code>abort()</code> / <code>disconnected</code>	两端

一条 TCP 连接可以用五元组标识：协议、本机 IP、本机端口、对端 IP、对端端口。管理端一个 `listen` 套接字可以 `accept` 出很多根已连接套接字，每个用户端一根。

5.2 阻塞 vs 事件驱动

教科书里 `recv()` 会卡住线程直到有数据。Qt GUI 不能卡死界面，所以用事件循环：

- 把套接字交给 Qt，有数据时发 `readyRead` 信号。
- 槽函数里一次性 `readAll()`，喂给 `Protocol::append`。
- 再 `while (hasPacket())` 取出完整 JSON。半包留在 `Protocol` 的缓冲里，等下次信号。

这就是「网络编程」在桌面程序里的标准形态：非阻塞 + 回调/信号。

6 本项目的 TCP 编程（8888）

6.1 服务端：先听再接

`adminserver/src/main.cpp` 在管理员登录框弹出之前就：

```
server.listen(QHostAddress::Any, 8888);
```

`Any` 表示所有网卡（0.0.0.0），所以局域网里别人才能连进来。若只听 127.0.0.1，组员跨机器会失败。听失败通常是第二份管理端还在抢端口。

每个新连接进入 `TcpServer::incomingConnection`：

1. 用内核给的描述符构造一根 `QTcpSocket`。
2. 每根线一个 `Protocol` 实例（缓冲不能混用）。
3. `readyRead`：拆包 → `Dispatch::handle` → `pack` 写回。处理是同步的：这一帧的 SQL 跑完才回包。作业并发量小，不用线程池。
4. `disconnected`：摘掉绑定；该用户已无其它线则 60 秒后释放桩。

6.2 客户端：拨号再发业务

`Client::connectTo` 先 `abort` 旧连接再 `connectToHost(host, port)`，保证一个 `Client` 最多一根 TCP。只有 `ConnectedState` 才 `write`，否则只报「未连接到服务器」，避免写半包。

`request` 组信封后 `Protocol::pack` 再 `write+flush`。`seq` 在本连接上自增。服务端推送的 `PUSH_CHARGE` 的 `seq` 固定为 0，与请求对不上号，界面按 `type` 区分。

6.3 接收循环为什么必须写成这样

```
codec_.append(sock_.readAll());
while (codec_.hasPacket())
    emit responded(codec_.nextPacket());
```

- 必须 `readAll`: 内核缓冲里可能已经粘了多帧。
- 必须 `while`: 一次信号可能拆出 0、1、多张 JSON。
- 半包不报错: 等下一轮 `readyRead`。

7 应用层协议: 我们自己的「HTTP」

TCP 只运字节。8888 上的语法是我们定的, 相当于一个极简 HTTP。

7.1 请求信封

```
{ "type": "LOGIN", "seq": 1, "role": "user", "token": "", "data": { ... } }
```

- `type`: 方法名 (类似 HTTP 的路径 + 动词合在一起)。
- `seq`: 这一帧的序号, 方便日志; 写操作还拿它做幂等键的一部分。
- `token`: 登录后的身份, 类似 Cookie / Authorization。
- `data`: 方法参数。

7.2 响应信封

```
{ "type": "LOGIN", "seq": 1, "code": 0, "message": "登录成功", "data": { ... } }
```

`code==0` 成功。失败时 `data` 为空, 避免界面把半成品当成功。常见码与 HTTP 故意对齐: 400 参数、401 未登录、403 禁止、404 没有、409 冲突。

7.3 请求-响应, 以及服务器推送

多数 `type` 是一问一答: 客户端发, 服务端立刻回同一 `seq`。充电中额外有**服务器推送**: `TcpServer` 每 5 秒对在线用户 `write` 一帧 `PUSH_CHARGE`, 客户端没有先发这个 `type`。这是 TCP 双向流才能做的事; 若改成短连接 HTTP, 就要改成客户端轮询或 SSE/WebSocket。

用户端仍每 1 秒自己问 `CHARGE_STATUS`: 推送丢了或旧客户端忽略推送时, 界面还能动。

7.4 鉴权发生在哪一层

`Protocol` 不看 `token`。`TcpServer` 只搬运。`RequestDispatcher` 查路由表: `LOGIN/REGISTER` 公开; 其余先 `SessionService::authenticate`。`Token` 是 UUID 字符串, 落 `session` 表, 30 分钟滑动过期。这是应用层会话, 不是 TLS 证书。

7.5 幂等: 网络编程里的业务问题

用户连点「开始充电」, 可能发出两帧相同 `type` 和 `seq`。TCP 保证都到达, 业务层必须认「这是同一操作」。键 `token|type|seq`, 8 秒内返回上次成功响应。这不是 TCP 重传, 是应用层防重复提交。

8 HTTP 与 HTTPS: 另一套应用层

8.1 HTTP 报文长什么样

浏览器和大屏、Qt 访问腾讯, 用的是 HTTP: 一行方法 + 路径, 然后头部, 空行, 可选正文。例如:

```
GET /api/overview HTTP/1.1
Host: 127.0.0.1:5000
User-Agent: ChargeHub/1.0 (campus training)
```

响应第一行是状态码 200 OK, 正文常是 JSON。Flask 的 `@app.get("/api/overview")` 就是在应用层注册这条路径。

HTTPS = HTTP 跑在 TLS 之上(传输仍是 TCP, 多了加密握手)。腾讯 `https://apis.map.qq.com` 由 Qt 网络库做证书校验; 业务代码只看到 URL 和 JSON。本作业 8888 没有 TLS, 局域网作业可接受; 不要把 token 打到公共 Wi-Fi 的演示里当安全课结论。

8.2 大屏: HTTP 只读

`dashboard/app.py` 监听 `0.0.0.0:5000`。三个 API 全是 GET, SQL 只有 SELECT。没有 token: 能访问 5000 的人只能看。这是有意的产品边界, 不是「忘了做登录」。

管理端用本机 TCP 探 5000 是否在听, 没有则拉起 Flask。这是「用 Socket 探端口」的典型用法, 不是用户协议。

8.3 出站 WebService

`TencentApi::signedUrl` 拼 `https://apis.map.qq.com + 路径 + 排序查询串`; 有 SK 时对 `path + "?" + query + SK` 做 MD5, 追加 sig。这是腾讯文档规定的签名, 不是 Socket 帧。

用户端 `QNetworkAccessManager::get` 是异步: 发完立刻回界面, 完成时槽函数读 `QNetworkReply`。管理端预测用 `fetchUrl`: 临时 `QEventLoop` 等到超时或完成, 会卡住调用线程约数秒, 所以不要放在拖地图的热路径里。

瓦片 URL 是普通 HTTP GET 图片。失败换列表里下一张(高德 → 腾讯)。Referer 头是防盗链要求, 属于 HTTP 头, 不是 TCP 选项。

8.4 DNS 与超时

`connectToHost("apis.map.qq.com")` 或 `QUrl` 会先 DNS 再连 443。校园网 DNS 慢或墙掉 OSM 时, 本项目已改走国内瓦片和腾讯, 避免依赖 `router.project-osrm.org`。所有出站带约 8 秒传输超时, 避免界面永久转圈。

9 连接生命周期与故障

9.1 正常关闭

用户关掉用户端 → TCP 断开 → 服务端 `disconnected`。若该用户没有其它用户端还连着, 开始 60 秒计时; 到点仍离线, 充电中订单改待结算、桩回闲置。这是应用层超时, 不是内核 TCP

keepalive 的默认 2 小时。

9.2 异常断电

对端崩溃可能既不挥手也不立刻被内核发现。所以有 HEARTBEAT（应用层保活并滑动 token）和 60 秒释放。仅靠 TCP 保活对「人走了桩还占用」反应太慢。

9.3 重连

connectTo 会 abort 旧线。Token 若仍在 30 分钟内，不必重新 LOGIN（「记住我」是本机设置自动再发 LOGIN，两套机制）。幂等缓存不跨进程持久化：管理端重启后连点保护从头算，token 仍可从库恢复。

9.4 一人两根线

同一账号两台电脑登录：两根 TCP，两份推送。释放桩看「是不是最后一根也断了」。这是会话设计，不是 TCP 本身的功能。

10 地址怎么填：联调时的网络

场景	用户端填写
本机自测	127.0.0.1:8888
组员连值班电脑	值班管理端底栏的 IP:8888
WSL 里听、Windows 里测	有时要用发行版 IP，不能想当然 127.0.0.1 一定转发
防火墙	服务器需放行 8888、5000；客户端做出站连接即可

虚拟机网卡要能互通（桥接或同一虚拟网）。NAT 只出站时，别人连不进你的 8888。这是网络层/部署问题，改 JSON 字段解决不了。

11 对照源码：建议阅读顺序

1. common/protocol.cpp: 长度、大端、粘包。可与 scripts/smokelocal.py 对照。
2. userclient/src/client.cpp: connectToHost、write、readyRead。
3. adminserver/src/main.cpp 的 listen，再 tcpserver.cpp 的 incomingConnection。
4. transport/requestdispatcher.cpp: 鉴权与幂等（应用层）。
5. services/sessionervice.cpp: token 与 30 分钟。
6. dashboard/app.py: HTTP GET 只读。
7. common/tencentapi.cpp: HTTPS URL、签名、同步/异步 GET、失败链路。

动手：先开管理端，再运行

```
python scripts/smokelocal.py
```

它用标准库 `socket.create_connection` 走完全相同的帧。能跑通，说明你理解的不是 Qt 特有魔法，而是 TCP + 长度前缀 + JSON。

12 把知识收成一张表

概念	本项目落点	常见误解
IP: 端口	127.0.0.1:8888 / 底栏 IP	用浏览器打开 8888
TCP 字节流	Protocol 切帧	一次 read 等于一条消息
大端长度	qToBigEndian / >I	按本机小端写长度
accept 多连接	每用户一个 QTcpSocket	一个 listen 只能服务一个人
事件驱动	readyRead	在 GUI 线程里阻塞 recv
应用层方法	JSON type	以为 TCP 知道 LOGIN
会话	token + session 表	把 TCP 连接当成永久身份
推送	服务端主动 write	以为只有客户端能说话
HTTP	大屏 5000、腾讯 HTTPS	把 8888 当成网站
失败隔离	天气失败不改订单	外网挂了就不能结算

答辩怎么讲见《最终答辩-网络通信与外部接口》；逐文件对照见《网络通信与外部接口说明》。